🛸 The Ultimate `uv` Master Guide (2026)

`uv` is an extremely fast Python package and project manager. It replaces `pip`, `pip-tools`, `venv`, and `pyenv` in a single, Rust-based binary.

🛠️ Step 1: Robust Installation

We prioritize the **Standalone Script** because it is self-contained and manages its own updates.

A. Windows (10/11) - The Robust Way

Open PowerShell as Administrator and run:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

B. Windows Alternative (WinGet)

If you prefer a package manager:

```
winget install --id astral-sh.uv -e
```

> ⚠️ ***If WinGet is missing:*** *Most modern Windows systems have it. If not, install the "App Installer" from the Microsoft Store, or run:* `Add-AppxPackage -RegisterByFamilyName -MainPackage Microsoft.DesktopAppInstaller_8wekyb3d8bbwe`

C. macOS / Linux

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

📄 Step 2: Understanding the "Brain" of your Project

Before running commands, you must understand the two files that `uv` creates. This is where most people get confused.

1. **`pyproject.toml` (The Manifest):** This is where **YOU** write. You list your dependencies here (e.g., `pandas`, `scikit-learn`). It defines the *intent* of your project.
2. **`uv.lock` (The Truth):** This is where **UV** writes. It contains the exact version and "hash" of every single package and sub-dependency. It ensures that if you share this project, your colleague gets the *exact* same environment down to the last byte.

🏗️ Step 3: Project Setup Sequence

Follow this exact order to migrate a project or start a new one.

1. Initialize & Pin

```
uv init                     # Creates the pyproject.toml scaffold
uv python pin 3.12          # "Pinning" locks the project to a specific Python version
```

> ***What is `uv python pin`?*** *It tells `uv` never to guess which Python to use. It writes the version into a `.python-version` file so everyone on the team uses the same engine.*

2. Migration (From old Pip)

If you are moving from an old project:

```
uv pip freeze > requirements.txt   # Capture current state
uv add -r requirements.txt         # Migrates them into pyproject.toml
```

3. The "Sync" Magic

```
uv sync
```

> ***What is `uv sync`?*** *This is the most important command. It looks at your `pyproject.toml`, calculates the `uv.lock`, and then **creates/updates the `.venv`** to match them perfectly. If you manually delete a folder in `.venv`, `uv sync` will bring it back.*

🔍 Step 4: Command Deep Dive

Command	What it actually does
`uv add <pkg>`	Adds the package to `pyproject.toml` and immediately updates your `.venv`.
`uv lock`	Updates the `uv.lock` file without changing your installed packages.
`uv run <file.py>`	Runs the script using the project's environment **without** needing to run `source activate`.
`uv venv --python 3.11`	Creates a virtual environment using a specific version (it downloads the version for you if missing).

📓 Step 5: Jupyter & VS Code Navigation

For Data Scientists, the biggest hurdle is getting the Notebook to "see" the environment.

The Robust Kernel Setup

1. **Install tools:** `uv add ipykernel`
2. **Register:**

   ```
   uv run python -m ipykernel install --user --name="my-env" --display-name="Python (UV-Project)"
   ```

3. **Confirm:** `jupyter kernelspec list` (This ensures the system knows where the kernel lives).

VS Code Navigation

1. Open your `.ipynb`.
2. **Top Right:** Click "Select Kernel".
3. **Selection:** Choose "Python Environments" -> Look for the one marked with your project name or `./.venv`.
4. **If it doesn't appear:** Press `Ctrl+Shift+P` -> `Developer: Reload Window`. This forced refresh is often required to pick up new `uv` paths.

⚡ Step 6: FAQ & Web Error Resolutions

❓ Issue: "uv: command not found" after installation.

Resolution: Your PATH wasn't updated.

- **Windows:** Look in `%USERPROFILE%\.local\bin`. Copy that path. Search "Env" in Start Menu -> Edit System Environment Variables -> Path -> Add the copied path.
- **Mac/Linux:** Add `export PATH="$HOME/.local/bin:$PATH"` to your `~/.zshrc`.

❓ Issue: SSL/Certificate Errors during download.

Resolution: Often happens in corporate environments (like Cognizant/Apple). Force `uv` to use system certificates:

- `set UV_NATIVE_TLS=true` (Windows)
- `export UV_NATIVE_TLS=true` (Mac/Linux)

❓ Issue: Kernel keeps "Detecting" or "Connecting" forever.

Resolution: This is usually a broken `ipykernel` installation within the `.venv`.

1. `uv pip install --force-reinstall ipykernel`
2. `uv run python -m ensurepip --upgrade`
3. Restart VS Code completely.

❓ Issue: How do I handle a "Project within a Project"?

Resolution: If you have a repo with multiple folders, `uv` might get confused. Ensure you run `uv init` in the **Root** and use `uv add` from there. If you need sub-project environments, use `uv venv <name>` inside the specific sub-folder.

❓ Issue: How to clear the cache when things get weird?

Resolution: `uv` caches everything to stay fast. If you suspect a corrupt package:

```
uv cache clean
```